Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Управление эксплуатацией систем ИИ 2026**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ПРАКТИЧЕСКАЯ РАБОТА №1

По дисциплине «Основы программирования»

Тема:

Выполнение программы, объекты и базовые типы Python

Выполнил Сорока Виктор Владленович
Проверил
Дата 29.09.2026

Санкт-Петербург 2026

Цель работы

Разобраться, как CPython выполняет программу, научиться отличать имя от объекта, корректно применять операции сравнения и работать с базовыми типами данных Python.

Ход работы

Задание 1. От исходного кода к байткоду.

Часть 1. Режимы запуска.

В интерактивном интерпретаторе (REPL) результат отображается автоматически (рисунок 1), потому что он предназначен для выполнения выражений и выводит результат выполнения последнего выражения из записанных. Принцип работы REPL описан в его названии: аббревиатура расшифровывается как «Read» (чтение ввода пользователя), «Eval» (обработка и выполнения кода), «Print» (вывод результата выполнения), «Loop» (повтор цикла).

```
# Выполнения выражения напрямую в REPL
$ python
Python 3.14.7 (main, Aug 10 2026, 07:46:56) [GCC 16.1.1 20260728] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3 * 4
14
```

Рисунок 1 — Выполнение выражения внутри интерактивного интерпретатора

В скрипте данное выражение не выводится. Для вывода необходимо использовать одну из функцй для вывода в системный поток «STDOUT» или «STDERR». На рисунке 2 показан результат выполнения скрипт после добавления функции «print()».

```
# После добавления print(...)
$ python task_1.py
14
```

Рисунок 2 — Результат работы скрипта после добавления функции вывода в системный поток

```
course = "Python"
hours = 4 * 2
print(f"{course}: {hours} часов")
```

Рисунок 3 — Код для анализа выражений и инструкций

Выражениями являются (рисунок 3): «"Python"», «4 * 2», «print(...)», «f"..."», «{course}», «{hours}». Они обязательно возвращают какое-либо значение (в том числе «None»). Инструкциями являются: course = "Python", hours = 4 * 2 (присваивания). Присутствующие литералы: «"Python"» (строка), «4» и «2» (целое число), «f"{course}: {hours} часов"» (f-строка). Создаются имена: «course» и «hours». Это наглядно видно в байткоде («STORE_NAME»).

Команда (рисунок 4) показывает версию установленного интерпретатора Python. На момент выполнения лабораторной работы версия 3.14.7 является последней стабильной (для хоста ОС Linux).

```
$ python --version
Python 3.14.7
```

Рисунок 4 — Команда для отображения версии интерпретатора

```
$ python -m ast task_1.py
Module(
   body=[
      # ...

      # Присваивание переменной course значения "Python"
      Assign(
         targets=[
            Name(id='course', ctx=Store())],
         value=Constant(value='Python')),

      # Присваивание переменной hourse результата выполнения выражения 4 * 2
      Assign(
         targets=[
            Name(id='hours', ctx=Store())],
         value=BinOp(
            left=Constant(value=4),
            op=Mult(),
            right=Constant(value=2))),

      # Вызов print()
      Expr(
         value=Call(
            func=Name(id='print', ctx=Load()),
            args=[
               JoinedStr(
                  values=[
                     FormattedValue(
                        value=Name(id='course', ctx=Load()),
                        conversion=-1),
                     Constant(value=': '),
                     FormattedValue(
                        value=Name(id='hours', ctx=Load()),
                        conversion=-1),
                     Constant(value=' часов')])])])])
```

Рисунок 5 — Пример абстрактного синтаксического дерева

AST (Abstract Syntax Tree) - абстрактное синтаксическое дерево - является представлением исходного кода программы, где каждый узел соответствует синтаксической конструкции языка. Построение дерева обычно является одним из следующих шагов после токенизации. Древовидная структура исходного кода полезна для кодогенерации, статического анализа кода и пр.

```
$ python -m dis task_1.py
  # ...
              # Загрузка константы "Python"
  7           LOAD_CONST               1 ('Python')
              STORE_NAME               1 (course)


              # Загрузка оптимизированной (вычисленной) константы 4 * 2
  8           LOAD_SMALL_INT           8
              STORE_NAME               2 (hours)

              # Загрузка функции print()
  9           LOAD_NAME                0 (print)
              # Добавление функции на стек
              PUSH_NULL
              # Загрузка переменной для f-строки
              LOAD_NAME                1 (course)
              FORMAT_SIMPLE
              # Загрузка константы f-строки
              LOAD_CONST               2 (': ')
              # Загрузка переменной для f-строки
              LOAD_NAME                2 (hours)
              FORMAT_SIMPLE
              # Загрузка константы f-строки
              LOAD_CONST               3 (' часов')
              BUILD_STRING             4
              CALL                     1
              # Выход из области выполнения функции
              POP_TOP
              # Результат функции print()
              LOAD_CONST               4 (None)
              RETURN_VALUE
```

Рисунок 6 — Пример байткода виртуальной машины CPython

Контрольный вопрос

Почему байткод CPython нельзя считать машинным кодом процессора?

Байткод - это высокоуровневое промежуточное представление кода. Он предназначен для выполнения не процессором, а интерпретатором. В отличие от машинного кода, он является по большей частью кроссплатформенным, но может зависить от версии интерпретатора. Байткод работает не с регистрами и сырой памятью, а с виртуальным стеком, кучей и динамической типизацией, что делает его гораздо более абстрактным, чем машинный код.

Задание 2. Имена, объекты и сравнение.

Часть 1. Ожидания от результата.

Запуск программы (рисунок 7) даёт следующий вывод (рисунок 8). Результат выполнения оправдывает все прелположения, данные в комментариям к коду программы.

```python
a = 1000
b = a
c = int("1000")

# Ожидаиие: int, int, int
print("Типы:", type(a), type(b), type(c))

# Ожидание: целые числа (identity хэш-коды объектов или адреса в памяти). Первы
print("Идентификаторы:", id(a), id(b), id(c))

# Ожидание: True, т.к. a = b = 1000 (сравнение по значению)
print("a == b:", a == b)

# Ожидание: True, т.к. a и b являются одним и тем же объектом в памяти (сравнен
print("a is b:", a is b)

# Ожидание: True, т.к. a = c = 1000 (сравнение по значению)
print("a == c:", a == c)

# Ожидание: False, т.к. a и c были созданы отдельно друг от друга, хотя имеют р
print("a is c:", a is c)
```

Рисунок 7 — Исходный код программы



Рисунок 8 — Результат работы программы

1. Связи между именами и объектами.
    1. Создаётся объект №1 типа int значения 1000. Создаётся имя a для хранения ссылки на объект №1.
    2. Создаётся имя b для хранения значения переменной a, то есть ссылки на объект №1 (имена a и b соответствуют объекту №1 в памяти).
    3. Создаётся объект №2 типа int значения 1000. Создаётся имя c для хранения ссылки на объект №2.
    4. Имена a, b, c указывают на объекты с одинаковыми значениями (int, 1000), но a/b и c хранят адреса разных объектов. Адрес, записанный в c не совпадает с адресом в a и b, потому что объект №2 создаётся во время работы программы (runtime) из строки при помощи int("1000").
2. Идентичность объектов сравнивает их адреса в памяти, а равенство значений либо сравнивает значения литералов (int, str, и т.д.), либо вызывает специальный метод __eq__, с помощью которого возможно переопределить оператор сравнения для объектов данного типа. Например, сделать возможным сравнение по значению объектов дата-классов (data class), представляющих собой иммутабельные модели данных.

3. Объект None типа NoneType является встроенной в язык константой (python -m dis task_2.py показывает LOAD_CONST 8 (NONE)), но в отличие от других типов данных может принимать только одно значение, что делает все созданные экземпляры идентичными между собой.
4. Выполним фрагмент в REPL (рисунок 9). Вывод доказывает, что конкретная реализация (CPython) может применять оптимизации перед запуском программы: после приведения константных выражений (слияние литералов строк) оптимизатор создаёт один объект и записывает его адрес в переменные first и second, что приводит к равенство значений и адресов, записанных в этих переменных.

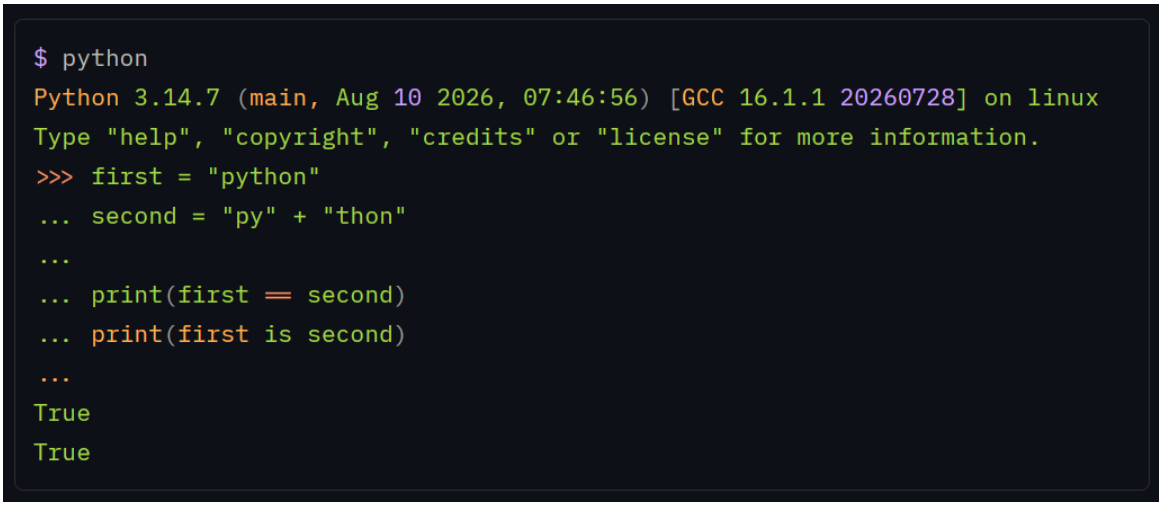


Рисунок 9 — Выполнение фрагмента кода в REPL

5. Применять «==» для сравнения объектов по значению (в большинстве случаев правильный подход), а «is» - для сравнения по ссылке: проверка на «None», проверка на уникальность для дефолтных значений параметров функции (sentinel), но нельзя использовать для чисел и строк (зависят от платформенной реализации).

Контрольные вопросы.

1. Создает ли инструкция «b = a» копию объекта?

Нет. В переменную b сохраняется адрес объекта, хранящийся в переменной a.

2. Что возвращают «type()» и «id()»?

Функция «type()» возращает тип объекта, переданного ей в качестве аргумента. Функция «id()» возращает адрес объекта, переданного ей в качестве аргумента, в памяти.

3. Почему None принято сравнивать с помощью «is», а числа и строки — с помощью «==»?

Потому что все объекты «None» являются единой константой - экземпляром типа «NoneType» и идентичны между собой, а сравнение по ссылке работает быстрее сравнения по значению в большинстве случаев.

Задание 3. Числовые типы, операции и преобразования.

Выполним программу (рисунок 10) и проанализируем результаты.

```
$ python ./task_3.py
Experiment A
x      = 17      (int)
y      = 5       (int)
x + y  = 22      (int)
x - y  = 12      (int)
x * y  = 85      (int)
x / y  = 3.4     (float)
x // y = 3       (int)
x % y  = 2       (int)
x ** y = 1419857 (int)

Experiment B
result = 107150860718626732094842504906000181056140481170553360744375038837035l

Experiment C
result                   = 0.30000000000000004   (float)
result == 0.3            = False                 (bool)
result - 0.3             = 5.551115123125783e-17 (float)
math.isclose(result, 0.3) = True                 (bool)

Experiment D
int("42")           = 42     (int)
float("3.14")       = 3.14   (float)
str(2026)           = 2026   (str)
bool(0)             = False  (bool)
bool(-1)            = True   (bool)
bool()              = False  (bool)
bool("False")       = True   (bool)
complex(2, -3)      = (2-3j) (complex)
complex(2, -3).real = 2.0    (float)
complex(2, -3).imag = -3.0   (float)
```

Рисунок 10 — Результат выполнения программы

«bool("False")» даёт истинное значение, потому что «"False"» - это не «bool», а непустая строка «str».

Контрольные вопросы.

1. Почему результат обычного деления двух объектов «int» имеет тип «float»?

Потому что результат деления целых чисел очень часто является дробным числом. Чтобы поведение функции было предсказуемым и тип результата не менялся динамически от конкретных чисел, у оператора / гарантированный возврат типа float. В Python в принципе нет общего предка для типов int и float, в отличие от других языков, например от Dart.

2. В чем отличие преобразования «int("42")» от «int(42.9)»?

«int("42")» выполняет парсинг строки: считывает текстовые символы и преобразует их в эквивалентное целое число. Если в строке будет точка, возникнет ошибка. «int(42.9)» выполняет отсечение дробной части у уже существующего числового типа «float», возвращая «42».

3. Какие значения из эксперимента D являются «falsy»?

«bool(0)» и «bool()» из эксперимента D, а также «bool("")» и «bool([])».

Задание 4. Строки, Unicode и форматирование.

Запустим программу и проанализируем результаты (рисунок 11).

```
$ python task_4.py
1.1. Первый символ имени:    А
1.2. Последний символ имени: а
2.1. Имя:     Анна
2.2. Фамилия: Смирнова
3.1. Строка в верхнем регистре: АННА СМИРНОВА
3.2. Строка в нижнем регистре:  анна смирнова
4. Инициалы: А.С.
5. Название курса в обратном порядке: nohtyP ан яинавориммаргорп ывонсО
6.1. Оператор %:     Анна Смирнова — Основы программирования на Python: 7/10 (7
6.2. Метод format(): Анна Смирнова — Основы программирования на Python: 7/10 (7
6.3. f-строка:       Анна Смирнова — Основы программирования на Python: 7/10 (7
7.1. Символ:          Я
7.2. Кодпоинт:        1071
7.3. Восстановленный: Я
7.4. UTF-8:           b'\xd0\xaf'
7.5. UTF-8 длина:     2
```

Рисунок 11 — Результат выполнения программы

Попытка изменить символ строки «student» приводит к следующей ошибке интерпретатора: «TypeError: 'str' object does not support item assignment». Данная ошибка возникает по причине того, что строковые объекты типа str являются неизменяемыми в Python. Внутренняя реализация CPython выделяет непрерывный буфер памяти под массив символов при инициализации строки и запрещает его модификацию без копирования (in-place): любая операция, изменяющая текст, требует создания нового объекта строки в памяти.

Контрольные вопросы.

1. Почему «len(symbol)» и «len(symbol.encode("utf-8"))» могут различаться?
   Функция len() для объекта str возвращает количество абстрактных символов Unicode (кодпоинтов). В случае символа “Я” кодовая позиция одна (1071 или U+042F), поэтому len("Я") == 1.
   Метод .encode("utf-8") переводит строку в последовательность байтов. UTF-8 является кодировкой переменной длины: ASCII-символы кодируются одним байтом, а кириллические символы требуют два байта: «b'\xd0\xaf'». Чем реже используется символ, тем длиннее его кодовое слово. Например, емодзи кодируются 4 байтами. Соответственно, количество байт (длина байтового массива) в строке, состоящей из одного символа «Я», закодированной в UTF-8, равно 2.
2. Что означают границы и шаг в записи «text[start:stop:step]»?
   «start» - индекс первого элемента среза (включается в результат). Если опущен, по умолчанию равен 0 при положительном шаге или концу строки при отрицательном. «stop» - индекс границы среза (не включается в результат). Если опущен, срез продолжается до границы строки. «step» - шаг смещения между индексами (по умолчанию 1). Отрицательный шаг задаёт обратный порядок обхода.
3. Почему строка считается неизменяемым объектом, хотя методы вроде .upper() возвращают измененный текст?
   В самом деле, метод «.upper()» не измененяет изначальный буффер в памяти, а создаёт новый объект с изменённой копией буфера оригинальной строки. Исходная строка остаётся полностью неизменной в памяти: записав строка в переменную до вызова

подобного метода, можно заметить, что после вызова метода по этому адресу остаётся оригинальная строка.

Задание 5. Информационная карточка вычислительного эксперимента.

Запустим написанный скрипт с тестовыми данными из примера, проанализируем результат выполнения (рисунок 12) и исходный код.

```
$ python task_5.py
Имя исследователя:
> Анна Смирнова
Название эксперимента:
> Проверка модели
Количество выполненных запусков (целое число):
> 8
Длительность одного запуска в секундах (вещественное число):
> 12.5
Действительная часть комплексного коэффициента (вещественное число):
> 2
Мнимая часть комплексного коэффициента (вещественное число):
> -3

==========================================
ЭКСПЕРИМЕНТ: Проверка модели
Исследователь: Анна Смирнова
Запуски: 8
Общее время: 100.00 с (1.67 мин)
Коэффициент: (2-3j)
Квадрат модуля: 13.00
Есть выполненные запуски: True
==========================================

Диагностическая информация:
- researcher_name → str: Анна Смирнова
- experiment_name → str: Проверка модели
- total_run_count → int: 8
- single_run_duration → float: 12.5
- complex_coefficient → complex: (2-3j)
- has_runs → bool: True
- total_run_duration → float: 100.0
- total_run_duration_minutes → float: 1.6666666666666667
- modulus_square → float: 13.0
```

Рисунок 12 — Результат выполнения программы с тестовыми данными из примера

1. После выполнения программы в области видимости остаются следующие имена.r
    1. esearcher_name (str): Анна Смирнова - введено пользователем, возвращается функцией input().
    2. experiment_name (str): Проверка модели - введено пользователем, возвращается функцией input().
    3. total_run_count (int): 8 - результат явного приведения строки к целому числу функцией int().
    4. single_run_duration (float): 12.5 - результат явного приведения строки к вещественному числу функцией float().
    5. complex_coefficient (complex): (2-3j) - результат явного приведения строки к вещественному числу функцией float().
    6. has_runs (bool): True - результат операции сравнения total_run_count > 0.
    7. total_run_duration (float): 100.0 - результат умножения целого числа на вещественное.
    8. total_run_duration_minutes (float): 1.6666666666666667 - результат операции обыкновенного (дробного) деления.
    9. modulus_square (float): 13.0 - результат сложения вещественных чисел, возведённых в степень.

2. Разбор строки: «modulus_square = complex_coefficient.real ** 2 + complex_coefficient.imag ** 2». Инструкция присваивания связывает результат вычисления (выражения) с именем modulus_square. Имя complex_coefficient ссылается на объект типа complex. Чтение свойств или вызов геттеров complex_coefficient.real, complex_coefficient.imag объекта complex_coefficient типа complex позволяют получить вещественную и мнимую составные части

комплексного числа, являющиеся вещественными числами float. Литерал «2» повторяется дважды, в обоих случаях служит показателем степени. Оператор «**» обладает наивысшим приоритетом в строке, и выполняет возведение числа в данную степень. Оператор «+» обладает самым низким приоритетом в данном выражении, и выполняет операцию сложения двух чисел. Оператор «=» обладает самым низким приритетом в данной строке, и выполняет операцию присваивания имени объекту (инструкция присваивания). Сначала выполняется «complex_coefficient.real ** 2», затем «complex_coefficient.imag ** 2». После этого операция сложения: «complex_coefficient.real ** 2 + complex_coefficient.imag ** 2». Последняя операция - присваивание: «modulus_square = complex_coefficient.real ** 2 + complex_coefficient.imag ** 2».

3. Путь программы.
    1. Интерпретатор читает исходный файл в кодировке UTF-8.
    2. Прозиводится токенизация исходного кода - разбор исходного текста на последовательную цепочку лексем (токенов): ключевые слова, операторы, идентификаторы, литералы, отступы и т. д.
    3. Парсер проверяет корректность последовательности токенов и строит AST (абстрактное синтаксическое дерево).
    4. Компилятор транслирует программу в набор инструкций для виртуальной машины — байткод.
    5. Виртуальная машина последовательно считывает и выполняет инструкции байкода.
    6. Функция input() всегда возвращает данные в виде строкового объекта (str), поэтому для выполнения специфичных операций (например, арифмитические вычисления с объектами int или float) необходимо выполнить явное преобразование в желаемый тип данных.

Контрольный вопрос.

После повторного присваивания нового значения уже существующему имени старого объекта уменьшится счётчик ссылок (при отсутствии ссылок объект помечается как доступный для сборщика мусора - GC), а новый объект будет создан (либо подгружен в память из кэша), и ссылка на него будет сохранена в переменную. Таким образом, повторное присваивание изменяет привязку имени, и оно начинает указывать на новый объект, а предыдущий объект не изменяется.

Вывод

В ходе выполнения лабораторной работы были исследованы модель выполнения программ на Python, механизмы CPython и объектная система языка.

Наиболее примечательными оказались три особенности.

1. **Адреса объектов и связывание имён.** Переменные в Python представляют собой именованные указатели на объекты в куче. В отличие от других высокоуровневых языков программирования (например, Java или Dart), Python не абстрагирует внутренний адрес объекта в памяти: в то время как другие языки позволяют получить хэшированное уникальное значение для каждого объекта (хэш-код), Python позволяет напрямую получить адрес объекта при помощи функции «id()». При этом сравнение по ссылке работает так же, как в других языках: результат операции «is» является истиной только для одного и того же объекта в памяти. Оператор сравнения «==» позволяет сравнивать объекты с разными адресами, и обычно выводит «True» для эквивалентных объектов.
2. **В Python 3 целые числа типа int обладают неограниченной разрядностью.** CPython динамически масштабирует представление в виде массива цифр, позволяя без переполнения вычислять большие значения.
3. **Строковые объекты типа str оперируют не байтовыми значенимя, а логическими единицами (кодовыми словами или кодпоинтами).** Преобразование строки в кодировку переменной длины, например UTF-8, создаёт байтовый массив, длина которого может отличаться от длины соотвествующей строки.